

Subject : Mobile Application

Topic : Displaying Maps.

Displaying Maps:-

Google maps is one of the many applications bundled with the android platform.

Creating the Project:-

- * To get started, need to first create an android project so that you can display google maps in activity.
- * Using android studio, create an android project and name it LBS.
- * From the Create New Project Wizard, select google maps activity.

Obtaining the Maps API Key:-

- * An application programming interface key (API key) is a unique code that is passed in to an API to identify the calling application or user.
- * API Keys are used to track and control how the API is being used.
- * Need to apply for a free google maps API Key before you can integrate google maps into your android application.

* To get a google maps key, open the google_maps_api.xml file that was created in your LBS project. Within this file is a link to create a new google maps key.

* Simply copy and paste the link into your browser and follow the instructions. Make note of the key that google gives you because you need it later in this project.

Displaying the map:-

* To create a new android project
File → New → New project → Google Maps Activity

* Using the project created in the previous section, replace the YOUR_KEY_HERE placeholder in the google_maps_api.xml with google maps key.

* Press Shift+F9 to debug the application on the android emulator then google maps displayed in the application's activity.

Displaying the Zoom Control:-

* The map to any desired location and it updates on-the-fly. You can enable users to zoom in or out of the map using the built-in zoom controls.

<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>

* Permission in AndroidManifest.xml.

Displaying the Built-In Zoom Controls:-

```
<fragment xmlns:android="http://schemas.android..."
    xmlns:tools="http://schemas.android.com/tools"
    xmlns:map="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/map"
    tools:context="com.ildimarzio.locationServices.Maps
        Activity"
    android:name="com.google.android.gms.maps.SupportMap
        Fragment"
    map:uiZoomControls="true"
/>
```

⇒ To observe the built-in zoom controls that appear at the bottom of the map when you click and drag the map. you can click the minus (-) icon to zoom out of the map, and the plus (+) icon to zoom into the map.

Program for zooming in or out of the map:-

MapsActivity.java.

```
import android.support.v4.app.FragmentActivity;
import android.os.Bundle;
import android.view.KeyEvent;
```



```
import com.google.android.gms.maps.CameraUpdateFactory;
import com.google.android.gms.maps.GoogleMap;
import com.google.android.gms.maps.OnMapReadyCallback;
import com.google.android.gms.maps.SupportMapFragment;
import com.google.android.gms.maps.model.LatLng;
import com.google.android.gms.maps.model.MarkerOptions;
```

```
Public Class MapsActivity extends FragmentActivity implements
OnMapReadyCallback {
```

```
    Private GoogleMap mMap;
```

```
@Override
```

```
    Protected void onCreate(Bundle savedInstanceState) {
```

```
        Super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_maps);
```

```
        SupportMapFragment mapFragment = (SupportMap
Fragment) getSupportFragmentManager()
```

```
            .findFragmentById(R.id.map);
```

```
        mapFragment.getMapAsync(this);
```

```
    }
```

```
@Override
```

```
    Public void onMapReady(GoogleMap googleMap) {
```

```
        mMap = googleMap;
```

```
        LatLng Sydney = new LatLng(-34, 151);
```



```
mMap.addMarker(new MarkerOptions().position(Sydney)
    .title("Marker is Sydney"));
```

```
mMap.moveCamera(CameraUpdateFactory.newLatLng(Sydney));
```

```
}
```

```
public boolean onKeyDown(int keyCode, KeyEvent event)
```

```
{
```

```
    switch (keyCode)
```

```
    {
```

```
        case KeyEvent.KEYCODE_3:
```

```
            mMap.animateCamera(CameraUpdateFactory.zoomIn());
```

```
            break;
```

```
        case KeyEvent.KEYCODE_1:
```

```
            mMap.animateCamera(CameraUpdateFactory.zoomOut());
```

```
            break;
```

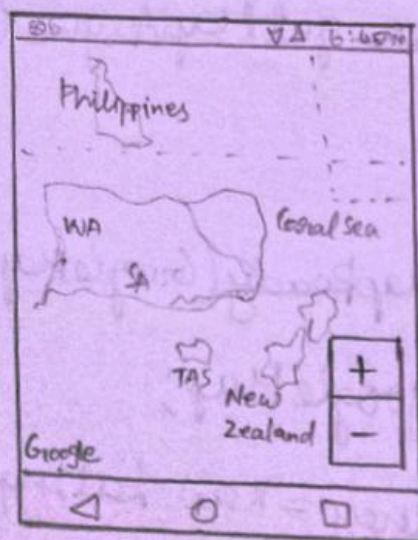
```
    }
```

```
    return super.onKeyDown(keyCode, event);
```

```
}
```

```
}
```

Design:



Changing Views:-

* Google maps is displayed in map view, which is basically drawings of streets and places of interest.

* Set google maps to display in satellite view using the `setMapType()` method of the `GoogleMap` class.

```
Public void onMapReady(GoogleMap googleMap) {  
    mMap = googleMap;  
    LatLng Sydney = new LatLng(-34, 151);  
    mMap.addMarker(new MarkerOptions().position  
        (Sydney).title("Marker in Sydney"));  
    mMap.moveCamera(CameraUpdateFactory.newLatLng  
        (Sydney));  
    mMap.setMapType(GoogleMap.Map_TYPE_SATELLITE);  
}
```

Navigating to a Specific Location:-

* Google Maps displays the map of Australia when it is first loaded. However, you can set Google maps to display a particular location.

* Use the `moveCamera()` method of the `Google Map` class.

Navigation the map to display a specific location:-

@Override

```
Public void onMapReady(GoogleMap googleMap) {
```

```
    mMap = googleMap;
```

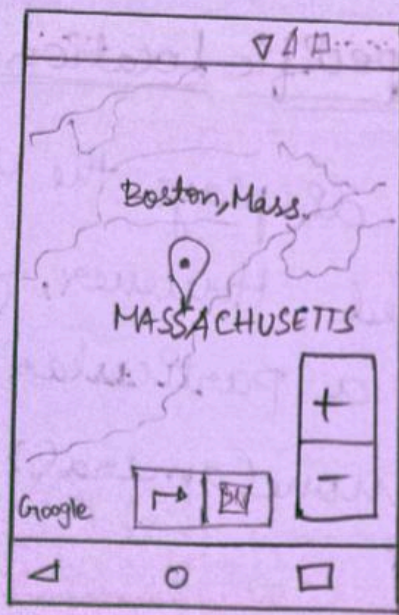
```
    LatLng boston = new LatLng(42.3601,  
                                -71.0589);
```

```
    mMap.addMarker(new MarkerOptions().position  
                    (boston).title("Boston, Mass"));
```

```
    mMap.moveCamera(CameraUpdateFactory.  
                    newLatLng(boston));
```

```
}
```

When the map is loaded, observe that it now animates to a particular location in Boston, Massachusetts.



Geocoding and Reverse Geocoding:-

* The latitude and longitude of a location, you can find out its address using a process known as reverse geocoding.

* Google Maps in Android supports reverse geocoding via the Geocoder Class. The code snippet shows how you can retrieve the address of a location just touched using the `getFromLocation()` method.

```
import android.location.Address;
import android.location.Geocoder;
import android.widget.Toast;
import android.os.Bundle;
import android.support.v4.app.FragmentActivity;

import java.io.IOException;
import java.util.List;
import java.util.Locale;

public class MapsActivity extends FragmentActivity
    implements OnMapReadyCallback {

    private GoogleMap mMap;

    @Override
    public void onMapClick (LatLng point) {
```



```
Geocoder geocoder = new Geocoder (getBaseContext(),  
Locale.getDefault());
```

```
try {
```

```
List<Address> addresses = geocoder.getFromLocation(  
Point.latitude, point.longitude, 1);
```

```
String add = "";
```

```
if (addresses.size() > 0)
```

```
{
```

```
for (int i = 0; i < addresses.get(0).  
getMaxAddressLineIndex();
```

```
i++)
```

```
add += addresses.get(0).getAddressLine(i) +  
"\n";
```

```
}
```

```
Toast.makeText (getBaseContext(), add, Toast.  
LENGTH_SHORT).show();
```

```
}
```

```
Catch (IOException e) {
```

```
e.printStackTrace();
```

```
}
```

```
}
```

```
});
```

```
}
```

```
}
```


* The Geocoder object converts the latitude and longitude into an address using the `getFromLocation()` method.

* After the address is obtained, to display it using the Toast Class that pin will not move. The address of a location that you touched on the map.

